

counterexample, remapped to the Skel line; contract hidden

Requirements

natural language

task text terminal line

The frozen contract is not in the prompt. It is read only by the Petri explorer.

LLM

proposes a skeleton

given

requirements

DSL grammar

last counterexample

not given: contract goals

prompt

.skel

Skel

skeleton DSL

mutex condvar semaphore channel atomic

lock spawn scope compute

```
fn t1() { lock a { lock b { compute "update"; }}}}
```

Checks and counterexamples are mapped back onto these lines.

lower + source map

ConcIR

sole semantics

Lowering is total: a checked skeleton becomes ConcIR plus a source map. The DSL adds no meaning.

validate support source map

ops

lock / unlock send / recv

wait / notify acquire / release



Petri exploration

bounded schedules

Unmet requirements

goal not reached

encode preserved behaviour missed

Concurrency bugs

deadlock

lock-order cycle

lost wakeup

atomicity violation

PASS counterexample UNKNOWN ≠ safe

PASS · LLM writes Rust

Rust

concrete program

On PASS, the LLM writes Rust from the accepted skeleton and the requirements.

verified Skel requirements

runtime concir_sync

Memory safety

Rust compiler

ownership

borrowes lifetimes

Safe Rust has no unchecked data races.

Concurrency safety

same Petri net

deadlock

lock order wakeup

Passed requirements bind the Rust code.

Instrumentation

while generating Skel

Probes ride along the skeleton so a run can be checked, not only the bounded net.

runtime trace schedule

Structural check

after Rust is emitted

The program is mapped back to a Petri net and compared with the verified one.

same locks same spawn

Generation accuracy

loom interleavings

miri undefined behavior

lockbud double-lock, order